

CHORUS: Foundation Models for Unified Data Discovery and Exploration

Moe Kayali
kayali@cs.washington.edu
University of Washington

Anton Lykov
alykov@cs.washington.edu
University of Washington

Ilias Fountalis
ilias.fountalis@relational.ai
RelationalAI

Nikolaos Vasiloglou
nik.vasiloglou@relational.ai
RelationalAI

Dan Olteanu
olteanu@ifi.uzh.ch
University of Zurich

Dan Suciu
suciu@cs.washington.edu
University of Washington

ABSTRACT

We apply foundation models to data discovery and exploration tasks. Foundation models are large language models (LLMs) that show promising performance on a range of diverse tasks unrelated to their training. We show that these models are highly applicable to the data discovery and data exploration domain. When carefully used, they have superior capability on three representative tasks: table-class detection, column-type annotation and join-column prediction. On all three tasks, we show that a foundation-model-based approach outperforms the task-specific models and so the state of the art. Further, our approach often surpasses human-expert task performance. We investigate the fundamental characteristics of this approach including generalizability to several foundation models and the impact of non-determinism on the outputs. All in all, this suggests a future direction in which disparate data management tasks can be unified under foundation models.

PVLDB Reference Format:
Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. CHORUS: Foundation Models for Unified Data Discovery and Exploration. PVLDB, 17(8): 2104 - 2114, 2024.
doi:10.14778/3659437.3659461

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at http://github.com/mkyl/CHORUS.

1 INTRODUCTION

Data discovery and exploration are major components of the workflow of analysts and data scientists. A survey conducted by the Anaconda data-science platform in 2021 found that analysts spend 40% of their working hours on data loading and cleaning [2]. Even with this colossal effort, 60-70% of data within an enterprise still goes unused for analytics [21], remaining as *dark data* [23, 63].

Recent developments in large language-models (LLMs) have unlocked human-level performance on diverse domain tasks. The discovery that these models can generalize to diverse domain-specific tasks that they have not been trained on [3, 26, 59, 60] has led to emergence of the term *foundation models* [5].

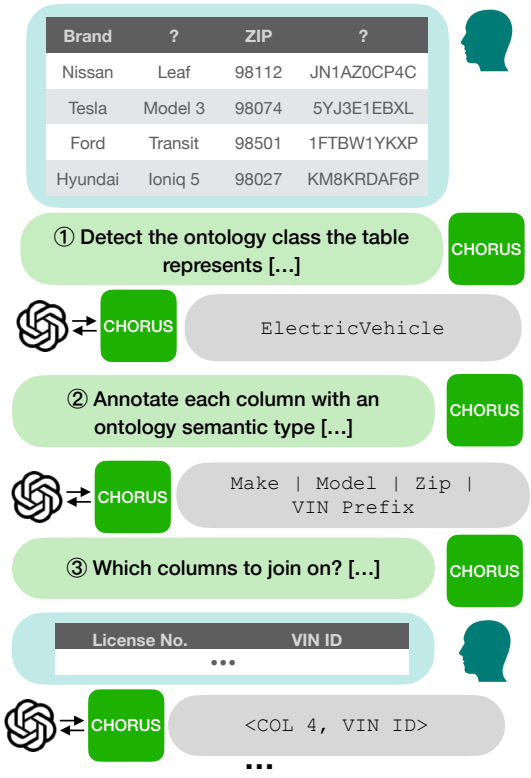


Figure 1: Data discovery tasks considered in this work. Given an ontology, such as DBPedia, ① we assign an overall type to the table and ② we annotate the columns with semantic types. Last, given another table, ③ we predict the join column. The user provides the data while CHORUS interacts with the foundation model. Data from [44], full prompts in Figure 3.

Despite their promise, serious risks have hampered the reception of foundation models. These include: spurious generation (including "hallucination") [24], factual recall limitations [39], bias [19], dataset contamination [14], logical shortcuts [50] and fallacies [38]. Naïve deployment can lead to unanticipated problems: it has already led to legal action [11] and recalls by major corporations [22]. These risks are now acknowledged by the creators of these models [6, 45, 54].

The goal of this paper is to demonstrate the utility of foundation models to the data discovery and exploration while mitigating

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659461

the aforementioned risks. We select three representative tasks to show the promise of foundation models: ① *table-class detection*, ② *column-type annotation* and ③ *join-column prediction*. An outline of our approach is shown in Figure 1. We call our approach CHORUS.

Contributions. We summarize our contributions:

- The first work to use foundation models for the data discovery tasks of table-class detection, column-type annotation and join-column prediction;
- Propose a novel system, CHORUS, whose flexible architecture enables the synthesis of multiple data discovery tasks and deploying risk mitigations;
- Design task-specific approaches that exploit zero- and few-shot strategies and allow information flow between tasks;
- Introduce the novel mitigation of *anchoring* to reduce foundation-model risks specific to this domain;
- Empirically validate CHORUS, comparing its performance with the state-of-the-art baselines across three individual tasks.

Discussion. Prior work has addressed these tasks individually. Landmark approaches like Sherlock [27] trained deep model architectures for a specific task, requiring 100K-1M labeled data points. More recent work such as DoDuo [52] and TaBERT [62] has focused on *representation learning*, learning embeddings for structured data by improving their performance on one or more downstream tasks.

Foundation models allow a substantially different approach: rather than the classical architecture where the outputs of the model are task-specific, the inputs and outputs of the model are natural language text. Training occurs not on tables or data management tasks specifically, but on general text. Performance on domain-specific tasks is solely by generalization.

This results in a high degree of flexibility. Novel tasks can be specified in natural text, without need for expensive data collection—task examples, metadata and constraints are all incorporated into the task easily. Another advantage of this approach is a **unified architecture**: tasks can utilize the overall context and previous outputs. For example, in Figure 1 the table class `ElectricVehicle` helps with deducing the outputs `Make`, `Model` in the next task.

Outline. Section 2 defines the three tasks investigated in this paper. Section 3 describes the architecture of CHORUS and key approaches. We evaluate the performance of CHORUS in Section 4's experiments. In that section, we also investigate the fundamental characteristics of this approach. We offer a discussion of those results in Section 5. This includes a discussion of promising future directions. Finally, we place this work within the literature in Section 6, discussing related works.

2 BACKGROUND

2.1 Tasks

We assume to be given a *data collection* consisting of a number of relational tables T_1,T_2,.... Each table T_i consists of a number of columns, or attributes, A_1,A_2,... and a number of rows, or tuples, r_1,r_2,... The name of a table T_i is, in general, non-informative, for example it may be simply a sequential ID. The columns may optionally have a name H_1,H_2,... or consist only of values.

In addition to the data collection, we are also given a reference ontology of table classes C_1,C_2,..., and a reference ontology of column types τ_1,τ_2,.... For example, the DBPedia.org types for the table classes include https://dbpedia.org/ontology/Actor and https://dbpedia.org/ontology/Continent and column types include https://dbpedia.org/ontology/areaTotal and https://dbpedia.org/ontology/birthDate.

We consider three tasks of interest on the data collection:

Definition 2.1 (① Table-class detection)**.** For each table T_i, determine its appropriate class C_j, such that every row r_1,r_2,... represents an instance of the C_j type. We adopt this definition from [33].

For example, table-class detection on the table given in Figure 1 could output `ElectricVehicle`, since each row of that table is an instance of that class. Alternatively stated, the table is about `ElectricVehicles`.

Definition 2.2 (② Column-type annotation)**.** For each table T_i, find a mapping from its attributes (columns) A_1,A_2,... to the reference column types τ_1,τ_2,..., such that each value in A_i is an instance of the τ_i type. See [1, 13].

For example, column-type annotation on the first column in Figure 1 could output `Manufacturer`, since the values are the respective manufacturers of each `ElectricVehicle`.

Definition 2.3 (③ Join-column prediction)**.** Assume an *execution log* L, a history of user actions including table joins and their join conditions, which maps many $(T_i,T_j) \rightarrow (A_k,A_l)$ where $A_k \in T_i, A_l \in T_j$. Given two tables T and T', with columns A_1,... and A'_1,... respectively, the *join-column prediction* task is to suggest a pair (A_k,A'_l) of columns such that the equality condition $A_k = A'_l$, which can be used to join the the tables, matches with the choice in the execution log L. For more discussion, see [61].

For example, given the table in Figure 1 and another table `car_registration(name, vehicle_id_number)`, join-column prediction could output `(VIN_prefix, vehicle_id_number)`. The correctness of the prediction depends on the ground truth of which columns the user did in-fact join on.

Ontologies Foundation models contain knowledge of ontologies such as DPBedia.org, Freebase and Wikidata. We focus on universal ontologies, that is, ontologies that aim to represent all entities in general. This is in-line with findings that foundation models encode highly technical knowledge, such as clinical reasoning [51] or electrical engineering principles [53].

3 APPROACH

We outline the structure of CHORUS in this section. First, we explore the core idea of ingesting relational data with foundation models and performing data exploration tasks in Subsection 3.1. Next, we describe the necessary post-processing and mitigations we develop in Subsection 3.2.

Figure 2 shows the architecture of the system. CHORUS has a unified architecture which runs multiple tasks in the same context, allowing for information flow. Each task is run sequentially, with the output of one task fed as context into future tasks.

For each task instance, CHORUS generates a prompt by concatenating six inputs: context, demonstration, data samples, metadata,

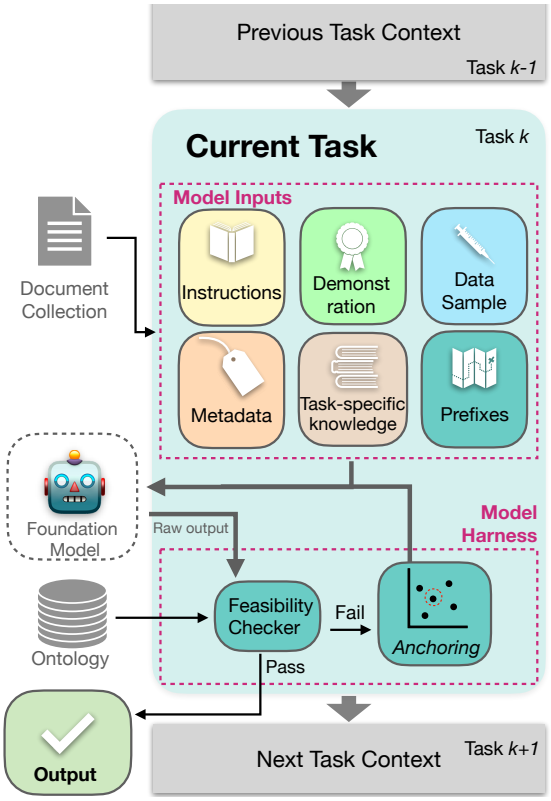


Figure 2: CHORUS system architecture.

task-specific knowledge, and prefixes. They form the "Model Inputs" box in Figure 2 and are color-coded so that they match the colored prompt components in Figure 3. This natural language input is then fed to the foundation model. The output is then subject to post-processing: checks of parsability and feasibility are conducted. If these pass, the output is extracted. Otherwise, we activate a mitigation process, called *anchoring*, in order to repair the error and prevent its propagation.

3.1 Model Inputs

We discuss what inputs are provided to the foundation model and how they are pre-processed and synthesized. We discuss the six components of the Model Inputs module in Figure 2, individually. These correspond to the six color-coded prompt components in Figure 3. Once generated, all the above inputs are concatenated to into a single prompt provided to the model.

Instructions. A description of the specific task (table-class detection, column-type annotation or join-column prediction) is provided to the foundation model in natural language. These are shown in yellow in Figure 3. For example, we translate the formal Definition 2.1 of the first task, table-class detection, into the English sentence "For the following CSV sample, select one DBpedia.org ontology that represents the dataset." For the third task, join-column prediction, we utilize a code-completion approach. We frame the task as code-completing a Pandas fragment that performs a join, with the code to complete shown in Figure 3c. We choose Pandas because it is a very popular framework, with more than millions of example lines of code

Legend: Instruction, Demonstration, Data sample, Metadata, Task-specific knowledge, Prefixes.

```
For the following CSV sample, select one DBpedia.org ontology that
represents the dataset from the following list:
AcademicJournal, AdministrativeRegion, Airline, Airport, [. . . ],
University, VideoGame, Work, Wrestler.
For example, for a dataset about hospitals, return
`https://dbpedia.org/ontology/Hospital`. Begin your answer with
'https://dbpedia.org/ontology'.
```
```
Brand, ␣, ZIP, ␣,
Nissan, Leaf, 98112, JN1AZ0CP4C,
Tesla, Model 3, 98074, 5YJ3E1EBXL,
[...]
```
```
```

(a) Table-class detection

```
Consider this example. Input:
```
```
Name, Famous Book, Rk, Year
Fyodor Dostoevsky, Crime and Punishment, 22.5, 1866
Mark Twain, Adventures of Huckleberry Finn, 53, 1884
Albert Camus, The Stranger, -23, 1942
```
```
Output:
`dbo:author, dbo:title, Unknown, dbo:releaseDate`.
For the following CSV sample, suggest a DBPedia.org Property for each
column from the `dbo:`namespace.
```[...]```
```

**(b) Column-type annotation**

```
Given two Pandas Dataframes, suggest what `pd.merge`
parameters to use to join the dataframes.
df1 =
```[...]```
df2 =
```[...]```
Complete the correct Pandas merge command. `pd.merge(df1, df2, left_on=
```

**(c) Join-column prediction**

**Figure 3: Prompts used in this paper, materialized with examples. Most prompt elements are fixed—only the data sample and metadata change for each instance.**

on the web. This is the *zero-shot prompt* setting: the model can be provided with instructions for a novel task and performs them directly.

*Demonstration.* For the first two tasks, we use the foundation models with task examples as an additional input: this is called the *few-shot prompt* setting. The model is given a few demonstrations of task completion, including inputs and outputs. This is shown in Figure 3 as green text.

*Data sample.* By serializing the input tables, we can input them into foundation models. For example, consider the example table from Figure 1 in the introduction. Serializing the table allows the foundation model to ingest the data. We use the comma-separated values (CSV) format, shown in blue in Figure 3.

Because the models have a limited context window size—typically in the few thousands of tokens—tables cannot always be ingested as a whole. Instead, we always serialize a sample of the rows. We find
```

a sample size of five is sufficient. Intuitively, it suffices to consider only a few values to determine column type.

Metadata. Schema information including column names (headers) and keys can be incorporated into the input, above the serialized data sample. We found that foundation models can adaptively infer whether the first column of the input is a header or data row, with no modification of the input required. This is shown in orange in Figure 3.

Task-specific knowledge. For some tasks, additional information can be used to guide the model. For ① table-class detection, if only certain output classes are desired, these can be listed to the model. The model will take these instructions into account when generating an output but they are not hard constraints. The encoding of such additional constraints for the table-class detection task is shown in Figure 3a.

Prefixes. We also provide the model with *prefixes* with which to complete. This includes the DBPedia format for the table-class detection task and a Pandas code fragment for the join-column prediction task. Both prefixes are highlighted in pink in Figure 3. Prefixes increase the likelihood the model will provide the output in a parsable format rather than deviating into a natural language description.

3.2 Model Harness

The foundation model is run within a harness that parses outputs into a symbolic representation and mitigates errors.

Constraint checks. Because the model is not constrained in its outputs, it may not always output a feasible answer. In this setting we impose three constraints: table types must belong to the ontology classes, column types must belong to the ontology properties and joins must be on existing columns. An output is infeasible if, in particular, it is not parsable or if it violates any of the three constraints. If this occurs, CHORUS performs anchoring.

Anchoring. If the constraints are violated, we do not simply move on to the next task. The risk is of hallucination snowballing [65]: once a foundation model makes a single spurious generation, subsequent outputs are more likely to also be wrong. The model will make mistakes it would otherwise be able to avoid. For example, in Figure 4(a): once nonexistent class `iucnStatus` is suggested, another nonexistent class `animalName` follows. Because we maintain context across tasks, we are particularly vulnerable to this.

We call the novel domain-specific mitigation we deploy *anchoring*, shown in Figure 4(b). CHORUS ends the conversation when an error is detected. It then initiates a new conversation, feeding the LLM with a false history in which the LLM did not hallucinate. This is possible because the conversational LLM takes as input the full history text, which we can retroactively modify. We insert artificially an existing class from the ontology (e.g. the nearest neighbor in the embedding space to the non-existing class). Fed with this cleaner input, the model is able to directly provide the correct answer.

4 EXPERIMENTS

We empirically evaluate CHORUS on the three tasks defined in Section 2.1. For each task, we select a task-specific benchmark and compare with baselines representing the state of the art. Table-class

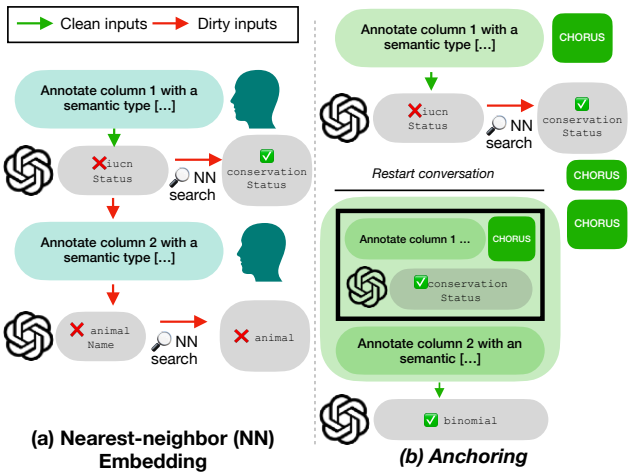


Figure 4: *Anchoring* illustrated. The LLM hallucinates an imagined label, `iucnStatus`. Under the standard approach, this poisons all the upcoming tasks; the nearest-neighbor post-processing cannot recover and outputs the incorrect label `animal`. With anchoring, CHORUS intervenes when the first error is detected. A new conversation is started and a *synthesized (false) history* is provided to the LLM, in which it did not make the mistake. With only clean inputs, LLM is able to correctly answer the next task correctly: `binomial`.

detection ① is evaluated in Section 4.1, ② column-type annotation in Section 4.2, and ③ join-column prediction in Section 4.3. The code for the experiments in available at https://github.com/mkyl/CHORUS.

Baselines. We considered the following state-of-the-art systems for data exploration: relevant systems include TABERT [62], DODUO [52], Sato [64], TURL [13], TaBBIE [29], Auto-suggest [61], Trifecta Wrangler [56], Paxata, Tableau Prep, and Sherlock. DODUO outperforms TURL and Sherlock on column-type annotation [52], so we select it for evaluation. Sato and Sherlock are similar, with Sato utilizing additional signals not found in our benchmarks, so we evaluate the better-established Sherlock. TaBBIE can embed tables but is not trained on column-type annotation unlike DoDuo and Tabert, so we avoid it for the column-type annotation task. TABERT is a work similar to DoDuo and TURL, but from the NLP community rather than the data management community, so we also test it too. For join-column prediction, Trifacta Wrangler outperforms Paxata and Tableau Prep [61]. Auto-Suggest is reported to outperform Trifacta Wrangler, but is a proprietary research project not released publicly. Thus we select Trifacta Wrangler for testing.

For the evaluated prior works TABERT, DODUO, Trifecta Wrangler and Sherlock [27, 52, 56, 62], we utilize each tool if applicable to the task. If the baseline is not designed for a particular task, but can be straightforwardly adapted, we do so. We describe all modifications in the task subsection and always use established adaptations if available. If the modifications required would be extensive enough to become their own research project, we consider that task unsupported. In all cases, we use the pretrained embeddings without modification, as provided by the authors. Table 1 outlines the systems we tested and tasks they support.

Table 1: *Capabilities of related systems.* Only our system supports all studied tasks out-of-the-box and without additional training.

System	Table-class detection	Column-type annotation	Join-column prediction
DoDuo [52]	●	✓	✗
Tabert [62]	●	✓	✗
Sherlock [27]	✗	✓	✗
Trifacta Wrangler [56]	✗	●	✓
Chorus	✓	✓	✓

✓ supported out-of-the-box, ✗ no support
● required modification or training data collection (see text)

Table 2: Summary of the datasets used in the paper. Numbers indicate the size of the data used.

Dataset Title	# Tables	Avg. # Columns	Avg. Rows
T2D-class v2	237	7.41	118
VizNet	~10 600	3.03	5 200
GitNotebooks	24 579	30.9	60 242
Overall	35 416	23.0	43 491

DoDuo provides two embedding variants: one trained on the WikiTables dataset and another on VizNet. We label them DoDuo-Wiki and DoDuo-Viz.

Datasets. Table 2 outlines the three experiment benchmarks we use. For the table-class detection task, we test on the T2D-class v2 dataset [48], a "gold standard" corpus of 237 tables, manually annotated by experts with one of 39 DBPedia.org classes. These tables were in turn selected from the Common Crawl corpus of web tables [17]. For column-type annotation, we sample a subset of the VizNet dataset [25], extracted by the Sherlock team [27], comprised of 32 386 columns with one of fifteen types from approximately 10 600 tables. This is in line with prior work that uses VizNet [28]. For the join-column prediction task, we use a dataset we call GitNotebooks, extracted by the Auto-suggest team [61]. We select 300 tables from that dataset for which we have join data to run manually. Here we use a sample as one of the baselines, Trifacta Wrangler, does not have an API but instead predictions must be produced manually. Separately, we run all 24 thousand tables on the baselines with an API. For the first two tasks, which require defining a type system for classes and properties, we use the DBPedia ontology [40] for our experiments. This is a community-sourced ontology and is the standard in previous studies.

Setup. We use the GPT-3.5 model [45] as it is the most widely-available large model with API access at the time of writing. All other code was run on a commodity laptop with 8 physical ARM cores and 16GB of main memory. Running all experiments came to a total of $20 in API costs.

We evaluate using the metrics *precision*, *recall* and F_1 score. Precision is the proportion of true positive results out of the total predicted positive results, while recall is the proportion of true positive results out of the total actual positive results in the dataset. The F_1 score is the harmonic mean of precision and recall. Since we deal with a multi-class setting, we calculate these metrics for each class separately then aggregate by taking the mean, weighted by the class size. Weighted precision, recall and F_1 are the standard metrics in prior work [7, 27, 52, 64]. We also report average throughput and cost for each task.

4.1 Table-class detection

For the first task, ① table-class detection, we tag each table with the DBPedia ontology entry that represents the row-type of the data. Of the 1 000 datasets that comprise the T2Dv2 dataset, 237 tables have table-class correspondences available while 763 do not—we exclude the unlabelled ones from the supervised evaluation. We call this subset of 237 annotated tables *T2D-class v2* and use it for evaluation on this task. We note that only 40 classes are utilized in this "gold standard" mapping, while DBPedia ontology has 769 classes.

We compare against the baselines DoDuo and TaBert. No approach in the prior work provides out-of-the-box capabilities on this task, so we add a classification layer on top of the pretrained embedding layer. After computing the column embeddings using DoDuo or TaBert, predictions are extracted by adding a pooling layer, fed to a multi-layer perceptron, and then finally taking the soft-max. This is a straightforward method of adapting the embeddings to our multi-class setting, used in prior benchmarks for table-class detection [33]. We fix the embeddings to their pretrained values and learn the weights of the classification layer using five-fold cross-validation.

Supervised variant. To allow for comparisons with prior work, we initially restrict our system to picking out of the 33 classes. This is because all other approaches require training on labelled instances—the baselines cannot predict outside those classes. We test 33 classes rather than 40 because the classes that occur only once cannot be tested on baselines that require supervised training (DoDuo and TaBERT), since a result requires a disjoint training and test set.

Table 3 shows the results. Chorus improves on the three baselines on all metrics. F_1 score is improved by 0.169 points, precision by 17.5 percentage points and recall by 15.5 percentage points. Of the baselines, DoDuo-Wiki provides the best F_1 and precision, while TaBERT provides the comparable recall. The best performing models, TaBERT and DoDuo-Wiki are trained on CommonCrawl, a superset of the T2Dv2 benchmark. DoDuo-Viz which is trained on the VizNet, a dataset disjoint from T2Dv2, has the weakest performance. The numbers for TaBERT are in line with prior replications [33], while to the best of our knowledge this is the first benchmarking of DoDuo on this task.

Unsupervised variant. Next, we relax the classification domain, allowing the foundation model to choose any of the 768 classes of the DBPedia ontology. We then compare the quality of the classes with that of the human-expert labels. DoDuo and TaBERT are not evaluated in this task setting as they cannot predict outside the classes they have observed in training.

For 93% of tables, our system produces correct results. Of that portion, 83 percentage points are comprised of exact matches, while 10 percentage points are *better-than-correct* results. This means we judge the predicted labels are clearly and unambiguously better than those selected by the benchmark authors. This is a strong claim so

Table 3: Weighted F_1 scores for *table-class detection* on T2Dv2 dataset. Systems are compared with the expert-annotated classes for each table. The $n = 237$ tables each correspond to one of 33 `DBPedia.org` classes.

	F_1-score	Precision	Recall
DoDuo-Viz	0.654	66.8%	68.3%
DoDuo-Wiki	0.757	78.6%	76.9%
TaBERT	0.746	76.3%	76.8%
Chorus	**0.926**	**96.1%**	**92.4%**

we list all such datasets in the technical report [31], with evidence. For the final 6% the answer is incorrect: this can mean the answer is wrong or simply worse than the label provided by the expert. This means that on the relations where chorus and the expert-label disagree, our system is 1.6× more likely to be correct.

Chorus has a throughput of nearly 31 tables per second on this benchmark and cost an average 2.5¢ per 100 table-class predictions.

4.2 Column-type annotation

Next, we compare the ability of our system to assign classes to table columns. VizNet is a collection of tables, extracted by the Sherlock [27] team from the VizNet repository [25] of data visualizations and open datasets. VizNet comprises 31 million columns in total, of which a test set of 142 000 can be used for evaluation—the rest have trained on by DoDuo and Sherlock. Of those, we select a subset of 15 classes which are supported by both DoDuo-Wiki and DoDuo-VizNet (these baselines support a disjoint set of classes), arriving at 32 386 test columns used as a benchmark in this section.

Baselines. We compare against TaBERT [62], DoDuo [52] and Sherlock [27] on this task. Since Sherlock and DoDuo are designed for column annotation, we use the out-of-the-box model provided by the original teams. We restrict both to the fifteen target classes by setting the probabilities of non-target classes to zero. For DoDuo-Wiki, which supports a distinct set of classes, we perform a manual mapping to the class names used by DoDuo-VizNet and Sherlock. For TaBERT we train an additional classification layer on top of the pre-trained embeddings that these frameworks provide. We fix the embeddings to their pretrained values and learn the weights of the classification layer using five-fold cross-validation.

Results. Table 4 contains the results for the VizNet dataset. Our fm-based approach improves performance on the measured metrics of F_1-score, precision and recall. The best performing method is Sherlock, narrowly beating DoDuo-VizNet, with a 0.954 F_1 score. If we consider methods which are not specifically pretrained on VizNet (note, which is also the test set) chorus is the best performing on all three metrics. It has comparable F_1 and precision to Sherlock, but 6 percentage points lower recall.

Note in particular DoDuo-Wiki, which does not have access to VizNet at pretraining time, has a large regression in performance compared to DoDuo-Viznet, nearly half F_1 points. This drop is in line with previous results, see Section 5. We sanity-check the low scores of TaBERT by replicating previously reported scores [33].

Table 4: Weighted F_1 scores for *column-type annotation* on VizNet test set, with $n = 32\,000$ columns. Systems are compared with the "gold standard" classes for each column. Methods which are also pre-trained on VizNet are marked with an asterisk $*$.

	F_1-score	Precision	Recall
DoDuo-VizNet$*$	0.876	89.4%	87.2%
Sherlock$*$	0.954	96.2%	94.6%
TaBERT	0.321	32.6%	32.0%
DoDuo-Wiki	0.440	59.2%	45.4%
Chorus	**0.891**	**91.2%**	**88.8%**

Chorus achieves a competitive throughput of 41 columns per second (col/s), comparable to Sherlock's 50 col/s and exceeding DoDuo's 7.3 col/s and TaBERT's 4.5 col/s. This corresponds to benchmark completion in 13 minutes, as contrasted with over 2 hours 10 minutes for TaBERT. The average cost of GPT-3.5 calls for this task was 1.3¢ cents per 100 columns.

4.3 Join-column prediction

Finally, we evaluate our approach's ability to suggest which columns are the correct choice for a join, the join-column prediction task. We use the *GitNotebooks* dataset from [61], a collection of 4 million Python notebooks (and their associated relational tables) including 24 thousand joins collected from Github. One of the baselines, Trifacta Wrangler, requires manual execution and recording of each prediction. For that reason we restrict this benchmark to 300 randomly sampled tables.

Baselines. For this task, we compare with three baselines. Jaccard similarity, J, is the first. Two columns are selected such that $\text{argmax}_{c \in C^T, c' \in C^{T'}} J(c,c')$ where $J(X,Y) = |X \cap Y|/|X \cup Y|$. This is a commonly used approach in the literature [10, 12, 43, 61]. Another baseline is Levenshtein distance [37], which selects the pair of column names with the smallest edit distance between them. The final baseline is Trifacta Wrangler [56], a commercial product spun off from the Wrangler research line [30]. When joining two tables in this product, it suggests the keys on which to join them. As no api was available, we obtain all Trifacta predictions by joining manually.

Results. Table 5 shows the quality of estimates for our approach and the baselines. We measure the quality of the predictions by the same criteria as the previous tasks. By these metrics, our approach improves the quality of predictions and beats the next-best approach by a clear margin: F_1 score is improved by 0.072, precision by 8.4 percentage points and recall by 6.0 percentage points. This performance is maintained when scaling to the full dataset. On this task, our system has an average throughput of 23.5 predictions per second and cost approximately 5¢ cents per 100 predicted joins.

4.4 Dataset contamination

Here we perform an experiment to validate whether any of the testing data occurred in the training corpus of the large-language model, an issue called *dataset contamination* or *data leakage*. Because

Table 5: F_1 scores, precision and recall for the *join-column prediction* task on GitNotebooks dataset.

	F_1-score	Precision	Recall
Manually-run subset, $n=300$			
Jaccard	0.575	60.7%	54.7%
Levenshtein	0.718	72.3%	71.3%
Trifacta Wrangler	0.823	82.6%	82.0%
Chorus	**0.895**	**91.0%**	**88.0%**
Full dataset, $n=24\,579$			
Jaccard	0.458	63.3%	35.9%
Levenshtein	0.777	78.7%	76.8%
Trifacta Wrangler		No API	
Chorus	**0.912**	**93.2%**	**89.4%**

Table 6: Data contamination experiment. Weighted F_1 scores for *table-class detection* on public benchmarks versus tables the foundation model is guaranteed to have not been trained on.

Dataset	F_1-score	Precision	Recall
Public benchmark (VizNet)	0.865	90.1%	86.7%
Guaranteed-unseen	0.857	90.0%	81.8%

these models are trained on internet data [18] and we use public benchmarks, they may have seen the test data in training.

We test on seven guaranteed-unseen tables (listed in the technical report [31]) and their columns, all uploaded between April–June 2023 to the federal data repository Data.gov. They are guaranteed-unseen because the foundation model training was completed on or before March 2023. Repeating the supervised column-type annotation task as in Section 4.2, we measure a 0.857 F_1 score, 90.0% precision and 81.8% recall. This is within 0.01 F_1 points, 0.1% precision and 5% recall of the benchmark results. See Table 6. The recall drop reflects the datasets being more diverse and therefore difficult to classify.

4.5 System characteristics

Determinism. We examine the impact of nondeterminism in the foundation model on the performance of Chorus. The randomness of the generation is controlled by the *temperature* hyperparameter. To assure that the results of Chorus are reliable, we conduct the following experiment: we run the T2D table-class detection benchmark 25 times, five trials for each value of T between 0,1/4,...,1. Figure 5 shows the result. Chorus's performance is consistent: at the ideal temperature setting the F_1 score sees error bars of 0.01 F_1 points. The best performance is obtained at lowest temperature, 0.0—this is in contrast to NLP tasks like summarization that benefit from higher temperatures. In the prior experiments we use the default temperature, as to get Chorus running with minimal hyperparameter tuning.

Alternative models. To demonstrate the versatility of this approach, we run Chorus with three alternative, open-source foundation models on the table-class detection task. We consider Vicuna [67], a variant of LLaMA [54] at two sizes: 13 billion parameters

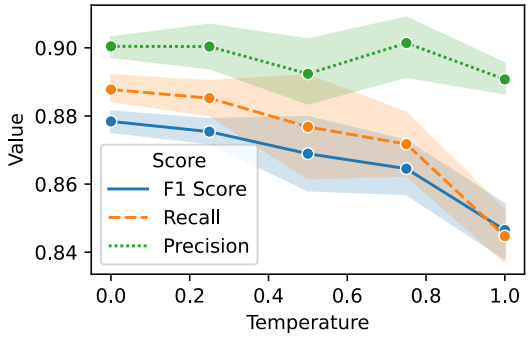


Figure 5: *Determinism vs. performance.* We conduct 25 runs of Chorus on the T2D table class benchmark. Shaded bands indicate confidence intervals. Temperature is a parameter controlling the randomness of the foundation model, with zero being the most (but not completely) deterministic.

Table 7: Alternative foundation models. Weighted F_1 scores for *table-class detection* on T2Dv2 dataset, for different choices of foundation model used by Chorus. Parameter size in brackets. GPT-3.5 numbers identical to experiment in Figure 3.

	Table-class correctness		
Model choice	F_1-score	Precision	Recall
GPT-3.5 (175B)	0.926	96.1%	92.4%
LLaMA 2 (70B)	0.893	92.2%	86.5%
Vicuna/LLaMA (13B)	0.713	79.2%	64.1%
Vicuna/LLaMA (7B)	0.713	75.3%	67.5%

and 7 billion parameters. The more advanced model is LLaMA 2 [55], the SOTA open-source model with 70 billion parameters.

Table 7 shows the results. While OpenAI's GPT model performs best, the best open-source model is very competitive. LLaMA 2 outperforms the best baseline model for this task—DoDuo-Wiki—by 0.136 F_1 points, on precision by 13.6 percentage points and on recall by 9.6 percentage points. This model lags behind the proprietary and larger GPT model by only a modest 0.03 F_1 points. Open-source LLMs are now compelling alternatives on the tested task.

Ablations. We conduct ablation experiments to measure the contribution of individual components of Chorus. We remove one component at a time and note the loss of scores compared to the unaltered model. Figure 6 shows the results. First, we remove the demonstration from the prompt. This results in an F1 score loss of 0.03, a recall loss of 4.7 and a precision loss of 4.7 percentage points. Next, we remove the metadata where it is available. This results in a cumulative F1 score loss of 0.04, a recall loss of 5.1 and a precision loss of 5.6 percentage points. After that, we disable anchoring. This results in a cumulative F1 score loss of 0.389, a recall loss of 47.4 and a precision loss of 31.9 percentage points. From this the prevalence of hallucinations can be gleaned: the substantive score loss implies that hallucination is highly prevalent. Finally, we remove the prefixes from the prompt. This results in a cumulative F1 score loss of 0.736, a recall loss of 92.3 and a precision loss of 53.1 percentage points.

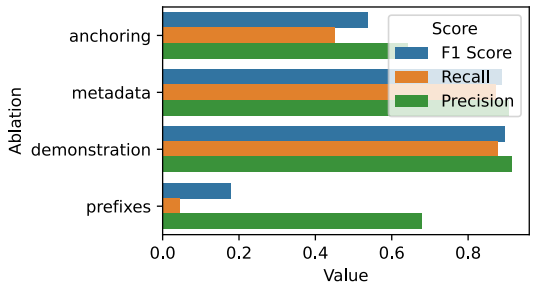


Figure 6: *Ablation experiments.* We ablate key features of CHORUS and report performance characteristics on the T2D table-class detection task.

5 DISCUSSION

Training data collection. A major advantage of a foundation-model approach is that there is no need for training on specific tasks. In contrast, TABERT requires 26 million tables for training its embeddings. In [13], the use of 250 labels for one task is considered a "small dataset" by the authors and leads to subpar performance. In contrast, our prompts in Figure 3 use zero or none examples for each task.

Out-of-domain performance. We note a troubling pattern of a lack of cross-domain generalization in representation-learning approaches. The tested baselines degrade when used to embed tables not from the dataset the embeddings were trained on. This finding is in-line with prior work: regressions of up to 0.40 and 0.30 F_1 points when generalizing to new datasets have been reported [13, 28].

Flexibility. Another advantage of CHORUS we observe in the experiments is task adaptability. In the ① table-class detection task, we are able to switch the prediction domain easily. Restricting to the 33 classes used by the benchmark can be done by providing the permitted classes to the foundation model; allowing the model to generalize to other DBPedia classes (the *unsupervised* heading of Section 4.1) is as simple as omitting those instructions. Contextual information, such as table title or URL, could be as easily added. Previously, such modifications would require retraining the embeddings.

Limitations and risks. We control the risk of dataset contamination by testing for it in Section 4.4. The performance of CHORUS on guaranteed-unseen datasets is comparable to those in public benchmarks, so good performance on the those benchmarks cannot be explained away as simple data contamination. Separately, formal linguistic fluency means that errors may fool human reviewers [35, 36]. This has been called *subtle misinformation* in prior work [47]. Finally, prompts may not be robust to changes [66].

Future directions: *Additional tasks.* The above hypothesis suggests the promising performance may extend to many more tasks. Related tasks to be explored include *schema auto-completion* [7], where missing parts of a partial schema are suggested to the user; *join-graph traversal*, where successive tables to join on are suggested [15]; and outlier detection, where erroneous data are detected. These are all promising because they involve composing simple patterns that are prevalent in FM's training data. On the other hand, novel approaches will likely be needed to apply FMs to tasks like *data provenance* [20], since these involving tracking more patterns than FM's are thought to currently support. Incorporating proprietary information may also be difficult to due the dearth of these patterns in the training data.

Private or domain-specific datasets. As with all the tested baselines, the foundation models are trained on public data. The distribution of data in the public sphere differs significantly from that in specialized domains or private data. It is worth investigating whether the observed capabilities continue to hold on e.g. enterprise data lakes. Further application to domain-specific ontology such as DRON, a pharmaceutical ontology of drugs, would also be a valuable investigation.

6 RELATED WORK

The seminal early work is WebTables [7], which extracts relational tables from web data, annotated with metadata for discoverability. This work introduced the related tasks: *schema auto-completion*, *attribute synonym finding*, and *join-graph traversal*. Early work on *wrapper induction* [34] also extracted tables from heterogeneous sources.

The promise of foundation models for data profiling was outlined in a recent position paper [58]. This paper was based on evidence of foundation models being able to predict correlations in data from the column names [57]. Another work considered foundation models for data wrangling [41]: comprising the tasks of entity matching, error detection and data imputation. Finally, most recently foundation models have been applied to the classic problem of wrapper induction in the system EVAPORATE [4].

The currently deployed generation of approaches has focused on representation learning. These include TURL [13], TABERT [62], DoDuo [52] and TABBIE [29]. These explore the use of fine-tuned language models for similar tasks. Prior to these table-embedding approaches, the prior generation of data tools involved data-intensive deep learning for specific tasks, e.g. Sherlock [27] and Sato [64].

Data discovery within data lakes is an active area of research, with recent works including: unionability search [32], joinability search [68], new index structures for faster correlated dataset search [49] and end-to-end systems for data ingestion and profiling [8]. Recent tutorials [16, 42] outline the prevalence of the problem of unstructured document data management. A user-study of scientists conclude that "current systems fail to sufficiently support scientists in their data-seeking process" [46]. One dataset-search survey [9] highlights key open problems: more natural query languages, better data integration, and incorporating external knowledge.

7 CONCLUSION

We propose CHORUS to integrate foundation models for data discovery. We show it provides superior performance on three exemplars: table-class annotation, column-type detection and join-column prediction. We conclude that foundation models hold promise as a core component of next generation data discovery systems.

ACKNOWLEDGMENTS

We thank Yejin Choi, Magdalena Balazinska, Cynthia Richey and Kyle Deeds. This material is based upon work supported by the National Science Foundation under Grants NSF-BSF 2109922 and NSF IIS 2314527.

REFERENCES

[1] Nora Abdelmageed, Jiaoyan Chen, Vincenzo Cutrona, Vasilis Efthymiou, Oktie Hassanzadeh, Madelon Hulsebos, Ernesto Jiménez-Ruiz, Juan Sequeda, and Kavitha Srinivas. Results of semtab 2022. In Vasilis Efthymiou, Ernesto Jiménez-Ruiz, Jiaoyan Chen, Vincenzo Cutrona, Oktie Hassanzadeh, Juan Sequeda, Kavitha Srinivas, Nora Abdelmageed, and Madelon Hulsebos, editors, *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2021, co-located with the 21st International Semantic Web Conference, ISWC 2022, Virtual conference, October 23-27, 2022*, volume 3320 of *CEUR Workshop Proceedings*, pages 1–13. CEUR-WS.org, 2022. URL https://ceur-ws.org/Vol-3320/paper0.pdf.

[2] Inc. Anaconda. State of data science. https://www.anaconda.com/resources/whitepapers/state-of-data-science-2021, July 2021.

[3] Jacob Andreas. Language models as agent models, 2022.

[4] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes, 2023.

[5] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ B. Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen Creel, Jared Quincy Davis, Dorottya Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, and et al. On the opportunities and risks of foundation models. *CoRR*, abs/2108.07258, 2021. URL https://arxiv.org/abs/2108.07258.

[6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4, 2023.

[7] Michael J. Cafarella, Alon Y. Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. Webtables: exploring the power of tables on the web. *Proc. VLDB Endow.*, 1(1):538–549, 2008. doi: 10.14778/1453856.1453916. URL http://www.vldb.org/pvldb/vol1/1453916.pdf.

[8] Sonia Castelo, Rémi Rampin, Aécio S. R. Santos, Aline Bessa, Fernando Chirigati, and Juliana Freire. Auctus: A dataset search engine for data discovery and augmentation. *Proc. VLDB Endow.*, 14(12):2791–2794, 2021. doi: 10.14778/3476311.3476346. URL http://www.vldb.org/pvldb/vol14/p2791-castelo.pdf.

[9] Adriane Chapman, Elena Simperl, Laura Koesten, George Konstantinidis, Luis-Daniel Ibáñez, Emilia Kacprzak, and Paul Groth. Dataset search: a survey. *VLDB J.*, 29(1):251–272, 2020. doi: 10.1007/s00778-019-00564-x. URL https://doi.org/10.1007/s00778-019-00564-x.

[10] Zhimin Chen, Vivek R. Narasayya, and Surajit Chaudhuri. Fast foreign-key detection in microsoft SQL server powerpivot for excel. *Proc. VLDB Endow.*, 7(13):1417–1428, 2014. doi: 10.14778/2733004.2733014. URL http://www.vldb.org/pvldb/vol7/p1417-chen.pdf.

[11] District Court. Mata v. avianca, inc. (1:22-cv-01461). Southern District of New York, New York, June 2023.

[12] Tamraparni Dasu, Theodore Johnson, S. Muthukrishnan, and Vladislav Shkapenyuk. Mining database structure; or, how to build a data quality browser. In Michael J. Franklin, Bongki Moon, and Anastassia Ailamaki, editors, *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, Madison, Wisconsin, USA, June 3-6, 2002*, pages 240–251. ACM, 2002. doi: 10.1145/564691.564719. URL https://doi.org/10.1145/564691.564719.

[13] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. TURL: table understanding through representation learning. *Proc. VLDB Endow.*, 14(3):307–319, 2020. doi: 10.5555/3430915.3442430. URL http://www.vldb.org/pvldb/vol14/p307-deng.pdf.

[14] Jesse Dodge, Maarten Sap, Ana Marasovic, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 1286–1305. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.98. URL https://doi.org/10.18653/v1/2021.emnlp-main.98.

[15] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. Deepjoin: Joinable table discovery with pre-trained language models. *CoRR*, abs/2212.07588, 2022. doi: 10.48550/arXiv.2212.07588. URL https://doi.org/10.48550/arXiv.2212.07588.

[16] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. Table discovery in data lakes: State-of-the-art and future directions. In Sudipto Das, Ippokratis Pandis, K. Selçuk Candan, and Sihem Amer-Yahia, editors, *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, Seattle, WA, USA, June 18-23, 2023*, pages 69–75. ACM, 2023. doi: 10.1145/3555041.3589409. URL https://doi.org/10.1145/3555041.3589409.

[17] Common Crawl Foundation. Common crawl, 2011. URL https://commoncrawl.org.

[18] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021. URL https://arxiv.org/abs/2101.00027.

[19] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, pages 3356–3369. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.301. URL https://doi.org/10.18653/v1/2020.findings-emnlp.301.

[20] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, PODS '07, pages 31–40, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595936851.

[21] Mike Gualtieri and Noel Yuhanna. *The Forrester Wave: Big Data Hadoop Distributions, Q1 2016*. Forrester Research, Inc., January 2016.

[22] Will Douglas Heaven. Why meta's latest large language model survived only three days online. *MIT Technology Review*, November 2022. URL https://www.technologyreview.com/2022/11/18/1063487/meta-large-language-model-ai-only-survived-three-days-gpt-3-science/.

[23] P. Bryan Heidorn. Shedding light on the dark data in the long tail of science. *Library trends*, 57(2):280–299, 2008.

[24] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL https://openreview.net/forum?id=rygGQyrFvH.

[25] Kevin Hu, Neil Gaikwad, Michiel Bakker, Madelon Hulsebos, Emanuel Zgraggen, César Hidalgo, Tim Kraska, Guoliang Li, Arvind Satyanarayan, and Çağatay Demiralp. Viznet: Towards a large-scale visualization learning and benchmarking repository. In *Proceedings of the 2019 Conference on Human Factors in Computing Systems (CHI)*. ACM, 2019.

[26] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *arXiv preprint arXiv:2201.07207*, 2022.

[27] Madelon Hulsebos, Kevin Zeng Hu, Michiel A. Bakker, Emanuel Zgraggen, Arvind Satyanarayan, Tim Kraska, Çagatay Demiralp, and César A. Hidalgo. Sherlock: A deep learning approach to semantic data type detection. In Ankur Teredesai, Vipin Kumar, Ying Li, Rómer Rosales, Evimaria Terzi, and George Karypis, editors, *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, pages 1500–1508. ACM, 2019. doi: 10.1145/3292500.3330993. URL https://doi.org/10.1145/3292500.3330993.

[28] Madelon Hulsebos, Çagatay Demiralp, and Paul Groth. Gittables: A large-scale corpus of relational tables. *CoRR*, abs/2106.07258, 2021. URL https://arxiv.org/abs/2106.07258.

[29] Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. TABBIE: pretrained representations of tabular data. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, pages 3446–3456. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.270. URL https://doi.org/10.18653/v1/2021.naacl-main.270.

[30] Sean Kandel, Andreas Paepcke, Joseph M. Hellerstein, and Jeffrey Heer. Wrangler: interactive visual specification of data transformation scripts. In Desney S. Tan, Saleema Amershi, Bo Begole, Wendy A. Kellogg, and Manas Tungare, editors, *Proceedings of the International Conference on Human Factors in Computing Systems, CHI 2011, Vancouver, BC, Canada, May 7-12, 2011*, pages 3363–3372. ACM, 2011. doi: 10.1145/1978942.1979444. URL https://doi.org/10.1145/1978942.1979444.

[31] Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. CHORUS: foundation models for unified data discovery and exploration. *CoRR*, abs/2306.09610, 2023. doi: 10.48550/arXiv.2306.09610. URL https://doi.org/10.48550/arXiv.2306.09610.

[32] Aamod Khatiwada, Grace Fan, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. SANTOS: relationship-based semantic table union search. *Proc. ACM Manag. Data*, 1(1):9:1–9:25, 2023. doi: 10.1145/3588689. URL https://doi.org/10.1145/3588689.

[33] Aneta Koleva, Martin Ringsquandl, Mitchell Joblin, and Volker Tresp. Generating table vector representations. *CoRR*, abs/2110.15132, 2021. URL https://arxiv.org/abs/2110.15132.

[34] Nicholas Kushmerick, Daniel S. Weld, and Robert B. Doorenbos. Wrapper induction for information extraction. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence, IJCAI 97, Nagoya, Japan, August 23-29, 1997, 2 Volumes*, pages 729–737. Morgan Kaufmann, 1997.

[35] Ellen J Langer, Arthur Blank, and Benzion Chanowitz. The mindlessness of ostensibly thoughtful action: The role of" placebic" information in interpersonal interaction. *Journal of personality and social psychology*, 36(6):635, 1978.

[36] David Langford. Comp.basilisk faq. *Nature*, 402(6761):465–465, 1999. doi: 10.1038/44964. URL https://doi.org/10.1038/44964.

[37] Vladimir Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. In *Soviet Physics Doklady*, volume 10, page 707, 1966.

[38] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.229. URL https://aclanthology.org/2022.acl-long.229.

[39] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories, 2022.

[40] Pablo N. Mendes, Max Jakob, and Christian Bizer. Dbpedia: A multilingual cross-domain knowledge base. In Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Mehmet Ugur Dogan, Bente Maegaard, Joseph Mariani, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Eighth International Conference on Language Resources and Evaluation, LREC 2012, Istanbul, Turkey, May 23-25, 2012*, pages 1813–1817. European Language Resources Association (ELRA), 2012. URL http://www.lrec-conf.org/proceedings/lrec2012/summaries/570.html.

[41] Avanika Narayan, Ines Chami, Laurel J. Orr, and Christopher Ré. Can foundation models wrangle your data? *Proc. VLDB Endow.*, 16(4):738–746, 2022. URL https://www.vldb.org/pvldb/vol16/p738-narayan.pdf.

[42] Fatemeh Nargesian, Erkang Zhu, Renée J. Miller, Ken Q. Pu, and Patricia C. Arocena. Data lake management: Challenges and opportunities. *Proc. VLDB Endow.*, 12(12):1986–1989, 2019. doi: 10.14778/3352063.3352116. URL http://www.vldb.org/pvldb/vol12/p1986-nargesian.pdf.

[43] Fatemeh Nargesian, Ken Q. Pu, Bahar Ghadiri Bashardoost, Erkang Zhu, and Renée J. Miller. Data lake organization. *IEEE Trans. Knowl. Data Eng.*, 35(1):237–250, 2023. doi: 10.1109/TKDE.2021.3091101. URL https://doi.org/10.1109/TKDE.2021.3091101.

[44] Washington State Department of Licensing. Electric vehicle population data electric vehicle population data, 04 2023. URL https://catalog.data.gov/dataset/electric-vehicle-population-data.

[45] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *CoRR*, abs/2203.02155, 2022. doi: 10.48550/arXiv.2203.02155. URL https://doi.org/10.48550/arXiv.2203.02155.

[46] Andrea Papenmeier, Thomas Krämer, Tanja Friedrich, Daniel Hienert, and Dagmar Kern. Genuine information needs of social scientists looking for data. *Proceedings of the Association for Information Science and Technology*, 58(1):292–302, 2021.

[47] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, H. Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant M. Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew J. Johnson, Blake A. Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Edward Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. Scaling language models: Methods, analysis & insights from training gopher. *CoRR*, abs/2112.11446, 2021. URL https://arxiv.org/abs/2112.11446.

[48] Dominique Ritze and Christian Bizer. Matching web tables to dbpedia - A feature utility study. In Volker Markl, Salvatore Orlando, Bernhard Mitschang, Periklis Andritsos, Kai-Uwe Sattler, and Sebastian Breß, editors, *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21-24, 2017*, pages 210–221. OpenProceedings.org, 2017. doi: 10.5441/002/edbt.2017.20. URL https://doi.org/10.5441/002/edbt.2017.20.

[49] Aécio S. R. Santos, Aline Bessa, Christopher Musco, and Juliana Freire. A sketch-based index for correlated dataset search. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pages 2928–2941. IEEE, 2022. doi: 10.1109/ICDE53745.2022.00264. URL https://doi.org/10.1109/ICDE53745.2022.00264.

[50] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *CoRR*, abs/2302.00093, 2023. doi: 10.48550/arXiv.2302.00093. URL https://doi.org/10.48550/arXiv.2302.00093.

[51] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Kumar Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble, Chris Kelly, Nathaneal Schärli, Aakanksha Chowdhery, Philip Andrew Mansfield, Blaise Agüera y Arcas, Dale R. Webster, Gregory S. Corrado, Yossi Matias, Katherine Chou, Juraj Gottweis, Nenad Tomasev, Yun Liu, Alvin Rajkomar, Joelle K. Barral, Christopher Semturs, Alan Karthikesalingam, and Vivek Natarajan. Large language models encode clinical knowledge. *CoRR*, abs/2212.13138, 2022. doi: 10.48550/arXiv.2212.13138. URL https://doi.org/10.48550/arXiv.2212.13138.

[52] Yoshihiko Suhara, Jinfeng Li, Yuliang Li, Dan Zhang, Çağatay Demiralp, Chen Chen, and Wang-Chiew Tan. Annotating columns with pre-trained language models. In *Proceedings of the 2022 International Conference on Management of Data*. Association for Computing Machinery, 2022. ISBN 9781450392495. URL https://doi.org/10.1145/3514221.3517906.

[53] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. Galactica: A large language model for science. *CoRR*, abs/2211.09085, 2022. doi: 10.48550/arXiv.2211.09085. URL https://doi.org/10.48550/arXiv.2211.09085.

[54] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023. doi: 10.48550/arXiv.2302.13971. URL https://doi.org/10.48550/arXiv.2302.13971.

[55] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307.09288.

[56] Trifacta. Trifacta wrangler. https://cloud.trifacta.com, 2023. Accessed: 2023-04-10.

[57] Immanuel Trummer. Can deep neural networks predict data correlations from column names? *CoRR*, abs/2107.04553, 2021. URL https://arxiv.org/abs/2107.04553.

[58] Immanuel Trummer. Towards nlp-enhanced data profiling tools. In *12th Conference on Innovative Data Systems Research, CIDR 2022, Chaminade, CA, USA, January 9-12, 2022*. www.cidrdb.org, 2022. URL https://www.cidrdb.org/cidr2022/papers/a55-trummer.pdf.

[59] Sai Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. Chatgpt for robotics: Design principles and model abilities. Technical Report MSR-TR-2023-8, Microsoft, February 2023. URL https://www.microsoft.com/en-us/research/publication/chatgpt-for-robotics-design-principles-and-model-abilities/.

[60] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models, 2022.

[61] Cong Yan and Yeye He. Auto-suggest: Learning-to-recommend data preparation steps using data science notebooks. In David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo, editors, *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, pages 1539–1554. ACM, 2020. doi: 10.1145/3318464.3389738. URL https://doi.org/10.1145/3318464.3389738.

[62] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. Tabert: Pretraining for joint understanding of textual and tabular data. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 8413–8426. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.745. URL https://doi.org/10.18653/v1/2020.acl-main.745.

[63] Ce Zhang, Jaeho Shin, Christopher Ré, Michael J. Cafarella, and Feng Niu. Extracting databases from dark data with deepdive. In Fatma Özcan, Georgia Koutrika, and Sam Madden, editors, *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, pages 847–859. ACM, 2016. doi: 10.1145/2882903.2904442. URL https://doi.org/10.1145/2882903.2904442.

[64] Dan Zhang, Yoshihiko Suhara, Jinfeng Li, Madelon Hulsebos, Çagatay Demiralp, and Wang-Chiew Tan. Sato: Contextual semantic type detection in tables. *Proc. VLDB Endow.*, 13(11):1835–1848, 2020. URL http://www.vldb.org/pvldb/vol13/p1835-zhang.pdf.

[65] Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. How language model hallucinations can snowball, 2023.

[66] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR, 2021. URL http://proceedings.mlr.press/v139/zhao21c.html.

[67] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *CoRR*, abs/2306.05685, 2023. doi: 10.48550/arXiv.2306.05685. URL https://doi.org/10.48550/arXiv.2306.05685.

[68] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. JOSIE: overlap set similarity search for finding joinable tables in data lakes. In Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska, editors, *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, pages 847–864. ACM, 2019. doi: 10.1145/3299869.3300065. URL https://doi.org/10.1145/3299869.3300065.